CSP-203: Software Tools & Technologies Lab



Plotting Tools: Matplotlib Advanced
(Lab Session – 17)

Dr. Anand M. Baswade
Indian Institute of Technology, Bhilai
anand@iitbhilai.ac.in

Contents

- ❑ Numpy
- ❑ Function Plot
- ❑ Multiplots
- ❑ Markers in Matplotlib
- ❑ GridSpec in Matplotlib
- ❑ HeatMap
- ❑ Image Plotting
- ❑ 3D Line Plot

NumPy (Numerical Python) is a core library for scientific and numerical computing in Python.

✅ Key Features:

- Works efficiently with large multi-dimensional arrays & matrices.
- Faster operations than Python lists.
- Has built-in mathematical, trigonometric, and statistical functions.
- Basis for scientific libraries like Pandas, SciPy, and Matplotlib.

Why use NumPy?

- Simplifies large dataset handling.
- Enables vectorized operations (no explicit loops).
- Optimized C backend for performance.

🧱 NumPy Array Creation and Basic Operations

NumPy provides several functions for creating arrays quickly:

- `np.array(list)` → Creates an array from a Python list.
- `np.zeros(shape)` → Creates an array filled with zeros.
- `np.ones(shape)` → Creates an array filled with ones.
- `np.eye(n)` → Creates an identity matrix.
- `np.arange(start, stop, step)` → Generates numbers with equal spacing.
- `np.linspace(start, stop, num)` → Generates evenly spaced numbers over a range.

Let's create and explore some arrays.

[ ]

```python
import numpy as np

# Creating arrays
arr1 = np.array([1, 2, 3, 4, 5])
arr2 = np.zeros((2, 3))
arr3 = np.ones((3, 3))
arr4 = np.eye(3)
arr5 = np.arange(0, 10, 2)
arr6 = np.linspace(0, 1, 5)

print("Array:", arr1)
print("Zeros:\n", arr2)
print("Ones:\n", arr3)
print("Identity:\n", arr4)
print("Arange:", arr5)
print("Linspace:", arr6)
```

```
Array: [1 2 3 4 5]
Zeros:
 [[0. 0. 0.]
 [0. 0. 0.]]
Ones:
 [[1. 1. 1.]
 [1. 1. 1.]
 [1. 1. 1.]]
Identity:
 [[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
Arange: [0 2 4 6 8]
Linspace: [0.   0.25 0.5  0.75 1.  ]
```

NumPy provides ready-to-use mathematical operations:

- Trigonometric: `np.sin(x)`, `np.cos(x)`, `np.tan(x)`
- Exponential: `np.exp(x)`
- Logarithmic: `np.log(x)`, `np.log10(x)`
- Square root: `np.sqrt(x)`

⚙️ Mathematical Functions in NumPy

NumPy provides element-wise operations on arrays, such as:

- `np.sin(x)` and `np.cos(x)` ⟶ Trigonometric functions.
- `np.log(x)` ⟶ Natural logarithm.
- These functions operate on each element of the array individually.

We'll also handle `log(0)` by using `(x + 1)` to avoid undefined values.

```
x = np.linspace(0, 2*np.pi, 5)
print("x values:", x)
print("sin(x):", np.sin(x))
print("cos(x):", np.cos(x))
print("log(x+1):", np.log(x+1))  # +1 to avoid log(0)
```

```
x values: [0.         1.57079633 3.14159265 4.71238898 6.28318531]
sin(x): [ 0.0000000e+00  1.0000000e+00  1.2246468e-16 -1.0000000e+00
 -2.4492936e-16]
cos(x): [ 1.0000000e+00  6.1232340e-17 -1.0000000e+00 -1.8369702e-16
  1.0000000e+00]
log(x+1): [0.         0.94421571 1.42108041 1.74263732 1.98556831]
```

Common Statistical Functions:

- Mean → `np.mean(array)`
- Median → `np.median(array)`
- Standard Deviation → `np.std(array)`
- Variance → `np.var(array)`
- Minimum/Maximum → `np.min(array)`, `np.max(array)`

[ ]

```
data = np.array([5, 10, 15, 20, 25])
print("Mean:", np.mean(data))
print("Median:", np.median(data))
print("Standard Deviation:", np.std(data))
print("Variance:", np.var(data))
```

```
Mean: 15.0
Median: 15.0
Standard Deviation: 7.0710678118654755
Variance: 50.0
```

Matrix Operations:

- Dot Product → `np.dot(a, b)`
- Matrix Multiplication → `np.matmul(a, b)`
- Inverse → `np.linalg.inv(matrix)`
- Determinant → `np.linalg.det(matrix)`
- Eigenvalues/Eigenvectors → `np.linalg.eig(matrix)`

🧠 Linear Algebra Operations in NumPy

NumPy includes a powerful linear algebra module `np.linalg` that supports:

- **Dot product:** `np.dot(A, B)`
- **Matrix inverse:** `np.linalg.inv(A)`
- **Determinant:** `np.linalg.det(A)`
- **Eigenvalues & Eigenvectors:** `np.linalg.eig(A)`

Let's perform these on sample 2×2 matrices.

```
A = np.array([[2, 1], [1, 3]])
B = np.array([[1, 2], [3, 4]])

print("Dot Product:\n", np.dot(A, B))
print("Matrix Inverse:\n", np.linalg.inv(A))
print("Determinant:", np.linalg.det(A))
values, vectors = np.linalg.eig(A)
print("Eigenvalues:", values)
```

```
Dot Product:
 [[ 5  8]
 [10 14]]
Matrix Inverse:
 [[ 0.6 -0.2]
 [-0.2  0.4]]
Determinant: 5.000000000000001
Eigenvalues: [1.38196601 3.61803399]
```

We can plot mathematical functions using NumPy for values and Matplotlib for visualization.

Steps:

1. Generate x values → `np.arange(start, stop, step)`
2. Compute y = f(x)
3. Use `plt.plot(x, y)` to visualize

Sine Function — Basic Line Plot

Purpose Plot a simple sine curve using `numpy` to create x-values and `matplotlib` to draw the line.

Key functions and syntax

- `np.arange(start, stop, step)`
  - Generates values from `start` (inclusive) to `stop` (exclusive) with the given `step`.
- `np.sin(x)`
  - Computes the sine of each element in array `x`.
- `plt.plot(x, y)`
  - Draws a 2D line plot of `y` vs `x`.
- `plt.title("...")`, `plt.xlabel("...")`, `plt.ylabel("...")`
  - Add title and axis labels.
- `plt.grid(True)`
  - Shows grid lines to improve readability.
- `plt.show()`
  - Renders the current figure (required in scripts; Colab usually auto-renders but keep it for clarity).

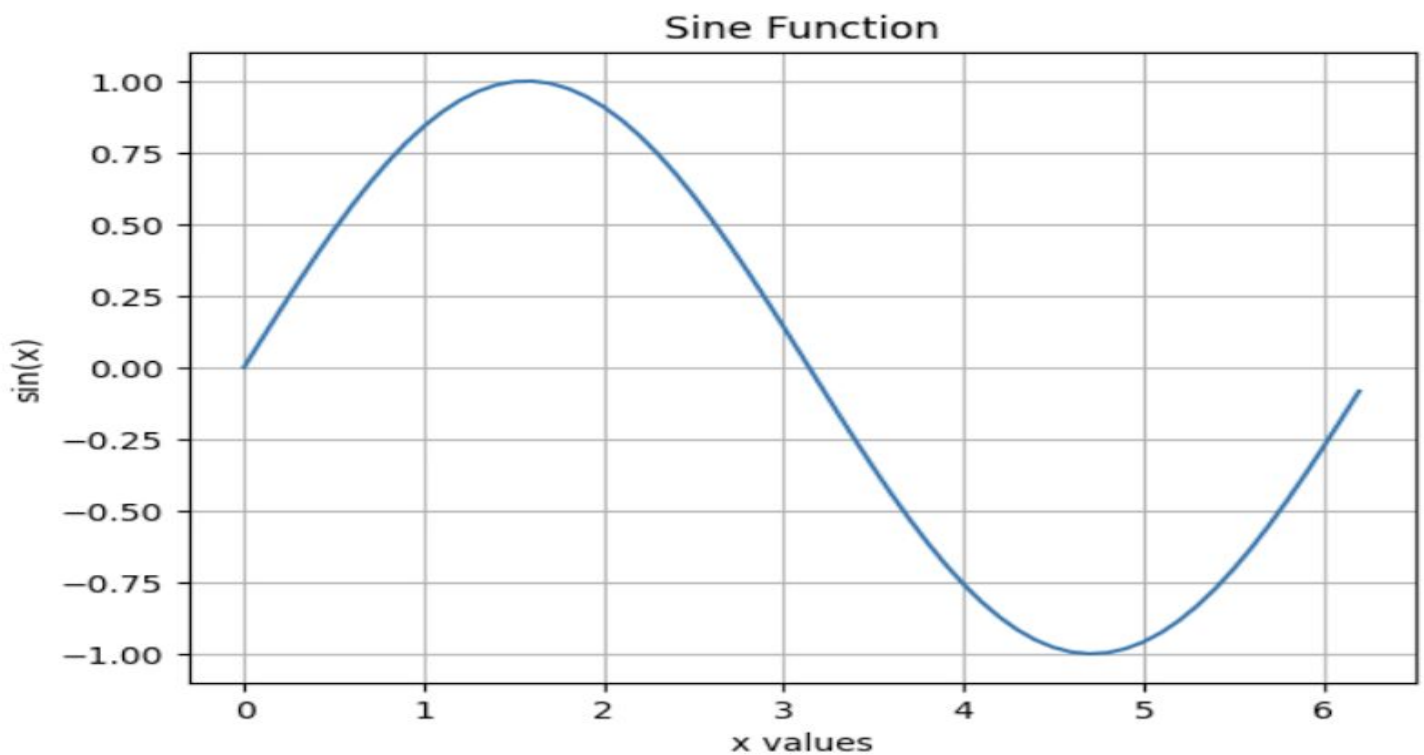
Line-by-line explanation of the code that follows

1. `x = np.arange(0, 2*np.pi, 0.1)`
   - Create x-values from 0 to `2π` (not including `2π`) with step 0.1.
2. `y = np.sin(x)`
   - Compute sine values for each x.

3. `plt.plot(x, y)`
    - Plot sine curve.
4. Add title and labels, enable grid, then `plt.show()` to display the plot.

```
import numpy as np
import matplotlib.pyplot as plt

x = np.arange(0, 2*np.pi, 0.1)
y = np.sin(x)
plt.plot(x, y)
plt.title("Sine Function")
plt.xlabel("x values")
plt.ylabel("sin(x)")
plt.grid(True)
plt.show()
```



Plot: y = sin(x) + log(x) + 1

Purpose Plot a combined function `y = sin(x) + log(x) + 1`. Demonstrates combining trigonometric and logarithmic functions and the need to avoid `log(0)`.

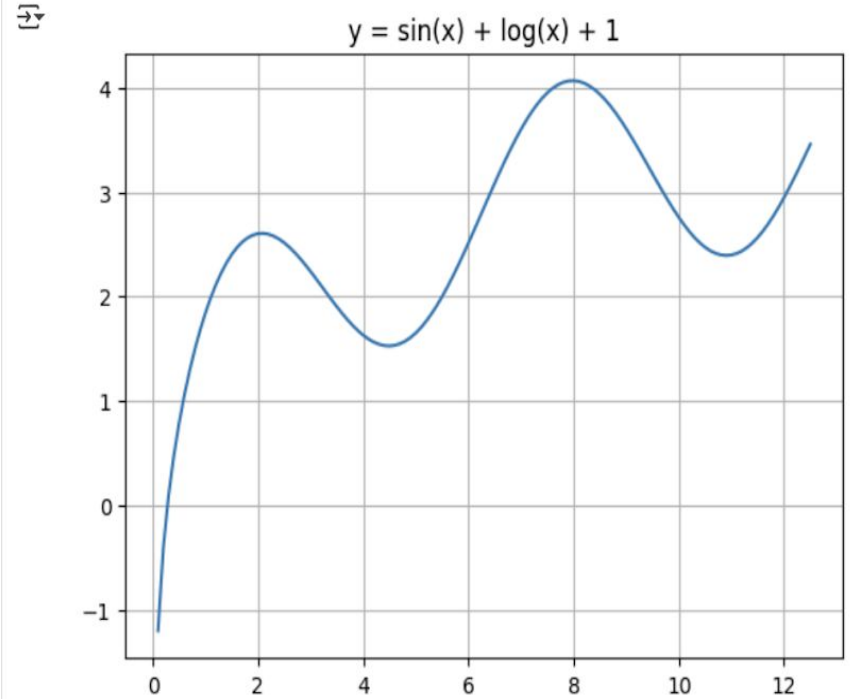
Key functions and syntax

- `np.arange(0.1, 4*np.pi, 0.1)`
  - We start at `0.1` (not 0) because `np.log(0)` is undefined.
- `np.log(x)`
  - Natural logarithm applied element-wise.
- `plt.plot()` and `plt.grid(True)` as before.

Line-by-line explanation

1. `x = np.arange(0.1, 4*np.pi, 0.1)`
   - `x` ranges from `0.1` to `~12.56` (approx `4π`). Start at `0.1` to avoid computing `log(0)`.
2. `y = np.sin(x) + np.log(x) + 1`
   - Compute the combined function element-wise.
3. Plot the function and show grid for readability.

```
x = np.arange(0.1, 4*np.pi, 0.1)
y = np.sin(x) + np.log(x) + 1
plt.plot(x, y)
plt.title("y = sin(x) + log(x) + 1")
plt.grid(True)
plt.show()
```



`plt.xticks()` and `plt.yticks()` are used to:

- Set tick positions
- Label them
- Rotate or format the labels

Sine Wave with Custom Ticks

Purpose Demonstrate how to replace numerical tick labels with descriptive labels on the x-axis and set explicit y-ticks.

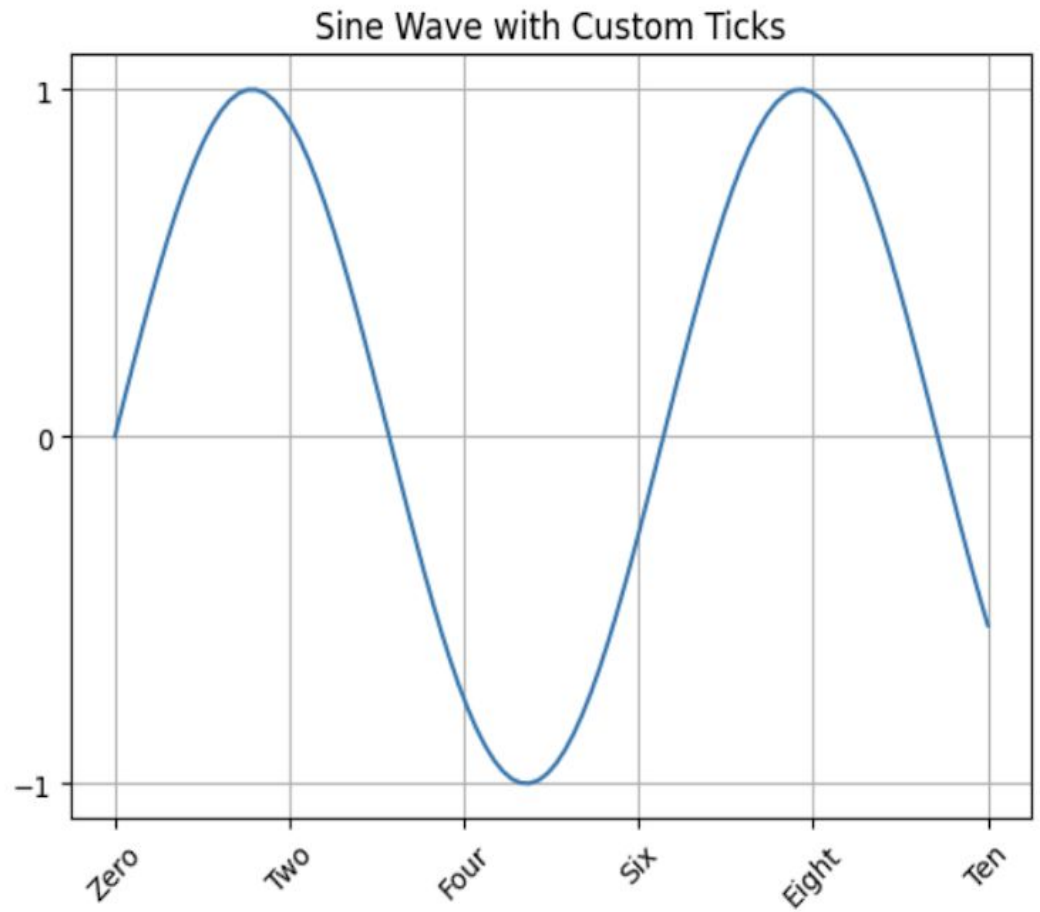
Key functions and syntax

- `np.linspace(start, stop, num)`
  - Generates `num` evenly spaced values from `start` to `stop` inclusive.
- `plt.xticks(ticks, labels, rotation=angle)`
  - `ticks` : positions where ticks should appear (list of numbers)
  - `labels` : list of strings shown at the positions
  - `rotation` : rotate labels to prevent overlap
- `plt.yticks([list])`
  - Set specific y-tick positions (and labels if passed a second list).
- `plt.grid(True)`
  - Show grid.

Line-by-line explanation

1. `x = np.linspace(0, 10, 100)` — 100 points from 0 to 10 inclusive.
2. `y = np.sin(x)` — sine values for plotting.
3. `plt.xticks([0,2,4,6,8,10], ['Zero','Two','Four','Six','Eight','Ten'], rotation=45)`
   - Replace numeric x-tick labels with words and rotate them 45°.
4. `plt.yticks([-1,0,1])` — set y-ticks at -1, 0, and 1.
5. Plot, title, grid, show.

```python
x = np.linspace(0, 10, 100)
y = np.sin(x)
plt.plot(x, y)
plt.xticks([0, 2, 4, 6, 8, 10], ['Zero', 'Two', 'Four', 'Six', 'Eight', 'Ten'], rotation=45)
plt.yticks([-1, 0, 1])
plt.title("Sine Wave with Custom Ticks")
plt.grid(True)
plt.show()
```



In Matplotlib, **markers** and **line styles** customize how data points and lines appear on a plot.

🎯 Marker Symbols

Markers are symbols that highlight each data point on a line.

Symbol	Marker Name	Description
`'o'`	Circle	Default circular marker
`'.'`	Point	Small dot
`','`	Pixel	Very small dot
`'x'`	X	Cross mark
`'+'`	Plus	Plus sign
`'*'`	Star	Asterisk/star symbol
`'s'`	Square	Square marker
`'D'`	Diamond	Regular diamond
`'d'`	Thin Diamond	Narrow diamond
`'v'`	Triangle Down	▼ Triangle pointing down
`'^'`	Triangle Up	▲ Triangle pointing up
`'<'`	Triangle Left	◀ Triangle pointing left
`'>'`	Triangle Right	▶ Triangle pointing right
`'h'`	Hexagon 1	6-sided polygon (type 1)
`'H'`	Hexagon 2	6-sided polygon (type 2)
`'p'`	Pentagon	5-sided polygon

🎨 Line Styles

You can modify the appearance of lines using the `linestyle` parameter:

Style	Description	Example
`'-'`	Solid line	Default style
`'--'`	Dashed line	– – – – – –
`'-.'`	Dash-dot line	–·–·–·–
`':'`	Dotted line	·····

You can also create **custom dash patterns** using tuples:

```
plt.plot(x, y, linestyle=(0, (3, 5, 1, 5)))
```

Line Styles in Matplotlib

Purpose Compare different line styles on the same plot to understand `linestyle` options.

Key syntax

- `plt.plot(x, y, linestyle='-')` — solid
- `linestyle='--'` — dashed
- `linestyle='-.'` — dash-dot
- `linestyle=':'` — dotted
- `linestyle=(0, (dash_length, gap_length, ...))` — custom dash pattern tuple
- `plt.legend()` — display legends for the plotted lines

Line-by-line explanation

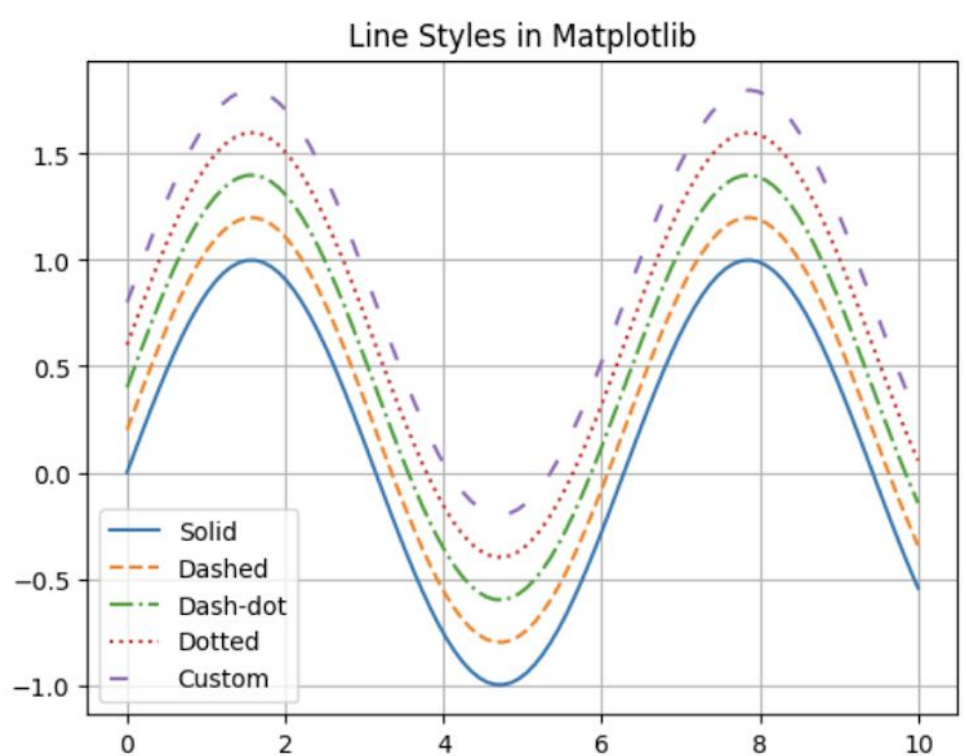
1. Compute `x` and `y`.
2. Plot the base sine curve with `linestyle='-'`.
3. Plot additional curves offset vertically (e.g., `y + 0.2`) with different `linestyle` for illustration.
4. Use `plt.legend()` to identify which line corresponds to which style.

```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 10, 100)
y = np.sin(x)

plt.plot(x, y, label='Solid', linestyle='-')
plt.plot(x, y + 0.2, label='Dashed', linestyle='--')
plt.plot(x, y + 0.4, label='Dash-dot', linestyle='-.')
plt.plot(x, y + 0.6, label='Dotted', linestyle=':')
plt.plot(x, y + 0.8, label='Custom', linestyle=(0, (5, 10)))

plt.legend()
plt.title("Line Styles in Matplotlib")
plt.grid(True)
plt.show()
```



Markers in Matplotlib

Purpose Show different marker styles (`'o'` , `'x'` , `'s'` , `'*'` , `'D'`) and how to combine markers with line styles.

Key syntax

- `marker='o'` — circle marker
- `marker='x'` — cross marker
- `marker='s'` — square
- `marker='*'` — star
- `marker='D'` — diamond
- `label='...'` — label for legend
- `plt.legend()` — show the legend

Line-by-line explanation

1. `x = np.linspace(0, 10, 10)` gives 10 points for clear markers.
2. `y = np.sin(x)` base signal.
3. Plot multiple curves each with different `marker` and `linestyle`.
4. Add legend, axis labels, grid, and show.

[2] 0s

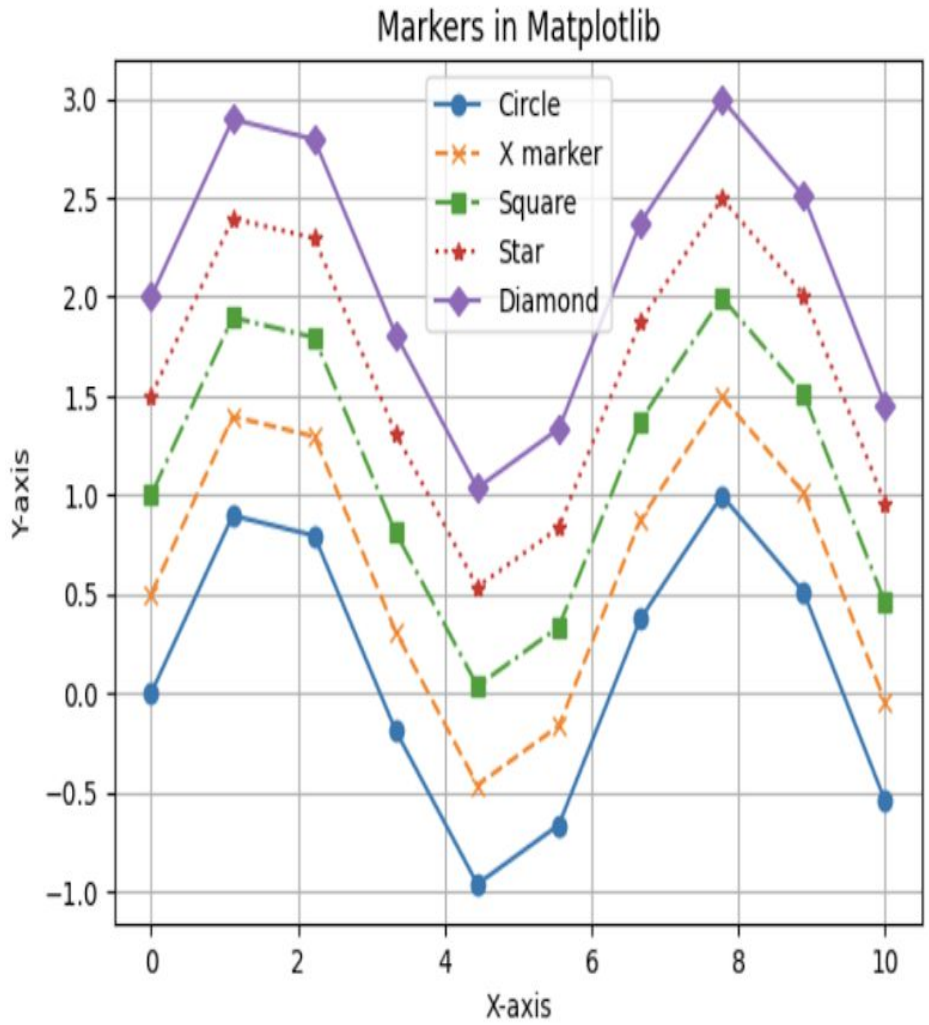
```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 10, 10)
y = np.sin(x)

plt.plot(x, y, marker='o', linestyle='-', label='Circle')
plt.plot(x, y + 0.5, marker='x', linestyle='--', label='X marker')
plt.plot(x, y + 1, marker='s', linestyle='-.', label='Square')
plt.plot(x, y + 1.5, marker='*', linestyle=':', label='Star')
plt.plot(x, y + 2, marker='D', linestyle='-', label='Diamond')

plt.title("Markers in Matplotlib")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.legend()
plt.grid(True)
plt.show()
```

0s



🧭 Multiple Plots & Visualization in Matplotlib 📊 1️⃣ Multiple Plots with Subplots The `plt.subplot()` function is used to create multiple plots (subplots) in a single figure.

Syntax:

```
plt.subplot(total_rows, total_columns, plot_number)


Visual Representation (1 Row × 2 Columns):

+------------------+------------------+
| subplot(1,2,1)   | subplot(1,2,2)   |
+------------------+------------------+


For 4 Subplots (2×2):

+-----------------+-----------------+
| subplot(2,2,1)  | subplot(2,2,2)  |
+-----------------+-----------------+
| subplot(2,2,3)  | subplot(2,2,4)  |
+-----------------+-----------------+
```

Grid of Subplots — 2 rows × 2 columns

Purpose Illustrate creating a subplot grid with `plt.subplots(2, 2)` and indexing with two indices: `axes[row, col]`.

Key syntax

- `fig, axes = plt.subplots(2, 2, figsize=(width, height))`
- `axes[0, 0].plot(...)` — top-left subplot
- `axes[1, 1].set_title(...)` — bottom-right subplot

Line-by-line explanation

1. Compute `x`.
2. Create a 2×2 axes array.
3. Plot different mathematical functions in each cell.
4. Use `plt.tight_layout()` to avoid label/title overlaps.

```
x = np.linspace(0, 10, 100)

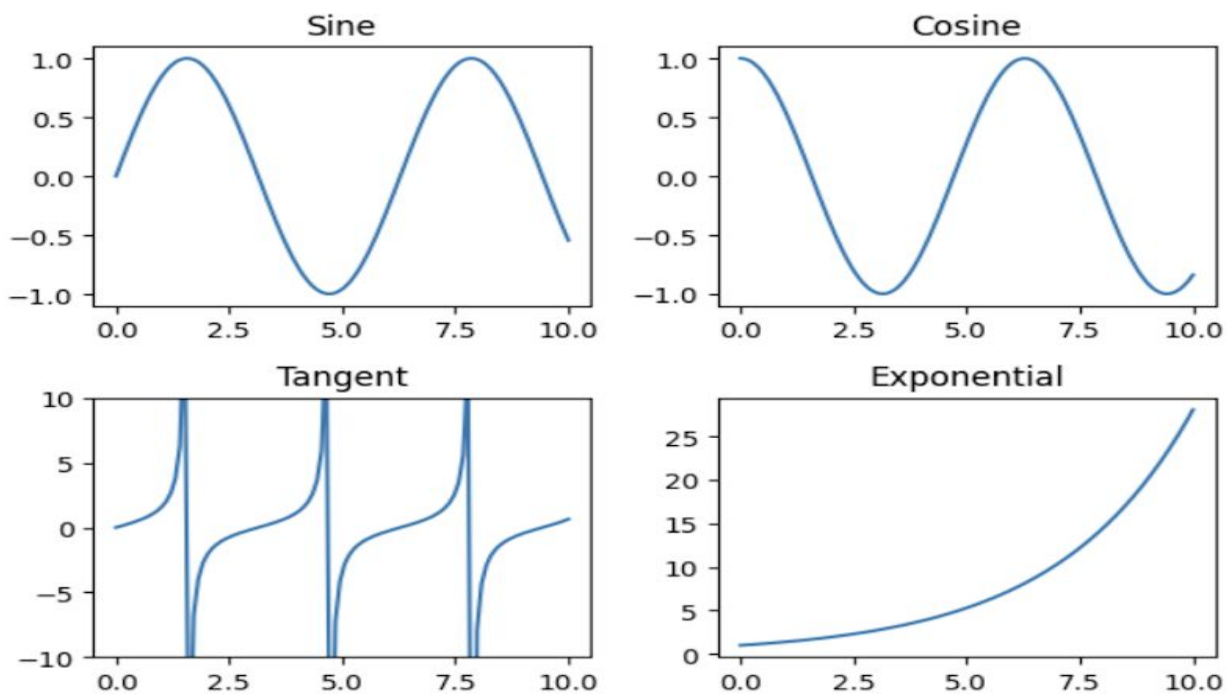
# Create 2x2 subplots
plt.subplot(2, 2, 1)
plt.plot(x, np.sin(x))
plt.title("Sine")

plt.subplot(2, 2, 2)
plt.plot(x, np.cos(x))
plt.title("Cosine")

plt.subplot(2, 2, 3)
plt.plot(x, np.tan(x))
plt.title("Tangent")
plt.ylim(-10, 10)

plt.subplot(2, 2, 4)
plt.plot(x, np.exp(x / 3))
plt.title("Exponential")

plt.tight_layout()
plt.show()
```



📈 2️⃣ Stacking Subplots (Vertical Layout) To create vertically stacked plots, use:

```
fig, axes = plt.subplots(4, 1, figsize=(6, 10))


4 → Rows, 1 → Column

axes → List of subplot objects (can be accessed via axes[i])

tight_layout() → Prevents label/title overlap
```

Multiple Subplots (4 rows × 1 column)

Purpose Demonstrate creating multiple vertically stacked subplots using `plt.subplots(nrows, ncols)` and accessing each axis via `axes[index]`.

Key syntax

- `fig, axes = plt.subplots(4, 1, figsize=(width, height))`
  - Returns `fig` (figure object) and `axes` (array of Axes objects).
- `axes[i].plot()` — plot on the `i`-th subplot.
- `axes[i].set_title("...")` — set title for that subplot.
- `axes[i].set_ylim(low, high)` — set y-axis limits for a subplot.
- `plt.tight_layout()` — automatically adjust spacing to prevent overlap.

Line-by-line explanation

1. Create an array `x` for plotting.
2. Create a figure with 4 stacked subplots (`axes[0]` to `axes[3]`).
3. Plot sine, cosine, tangent, and exponential on separate axes.
4. Use `set_ylim` on tangent to avoid extreme values dominating the layout.
5. `plt.tight_layout()` to make spacing neat, then show.

```
x = np.linspace(0, 10, 100)
fig, axes = plt.subplots(4, 1, figsize=(6, 10))

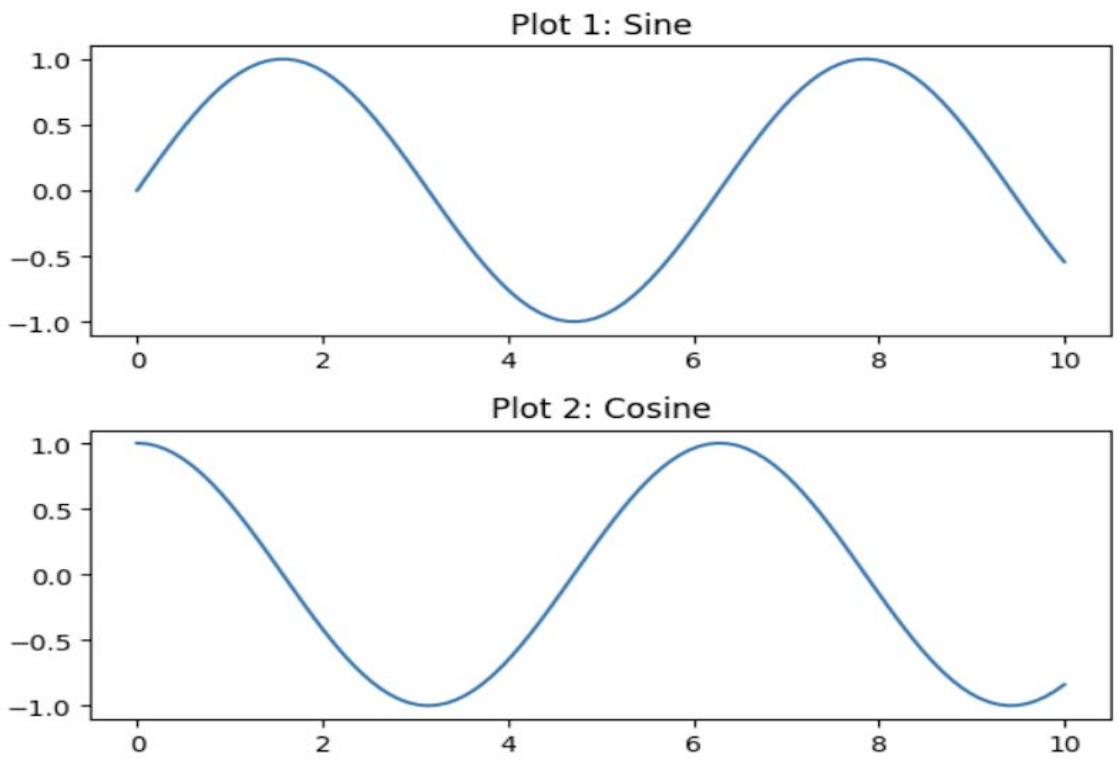
axes[0].plot(x, np.sin(x))
axes[0].set_title("Plot 1: Sine")

axes[1].plot(x, np.cos(x))
axes[1].set_title("Plot 2: Cosine")

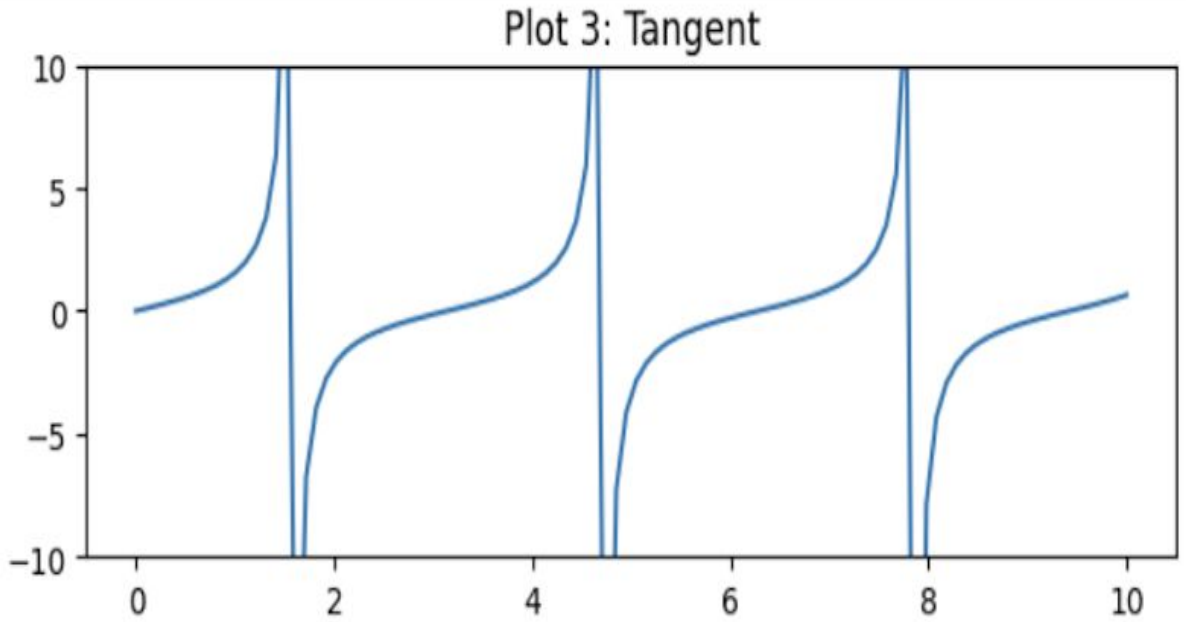
axes[2].plot(x, np.tan(x))
axes[2].set_ylim(-10, 10)
axes[2].set_title("Plot 3: Tangent")

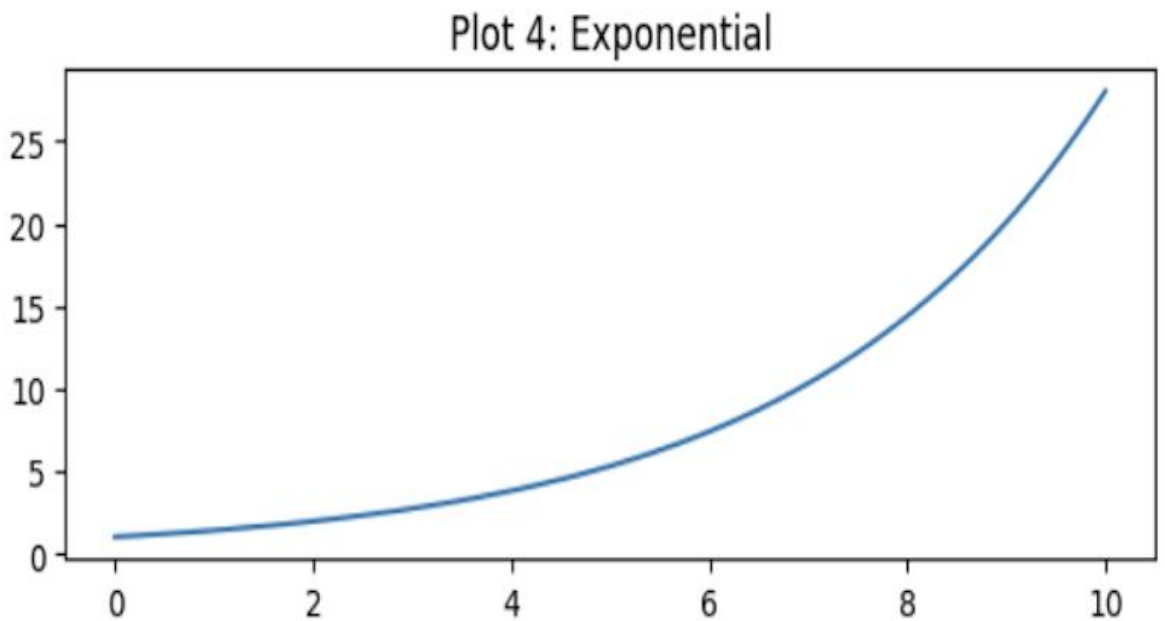
axes[3].plot(x, np.exp(x / 3))
axes[3].set_title("Plot 4: Exponential")

plt.tight_layout()
plt.show()
```



[↓]


Plot 3: Tangent
10
5
0
−5
−10
0
2
4
6
8
10


Plot 4: Exponential
25
20
15
10
5
0
0
2
4
6
8
10

3️⃣ 2×2 Subplots Using Axes Array `plt.subplots(2, 2)` returns a 2D NumPy array of Axes.

Access via:

- `axes[0, 0]` → top-left
- `axes[0, 1]` → top-right
- `axes[1, 0]` → bottom-left
- `axes[1, 1]` → bottom-right

```
x = np.linspace(0, 10, 100)
fig, axes = plt.subplots(2, 2, figsize=(10, 6))

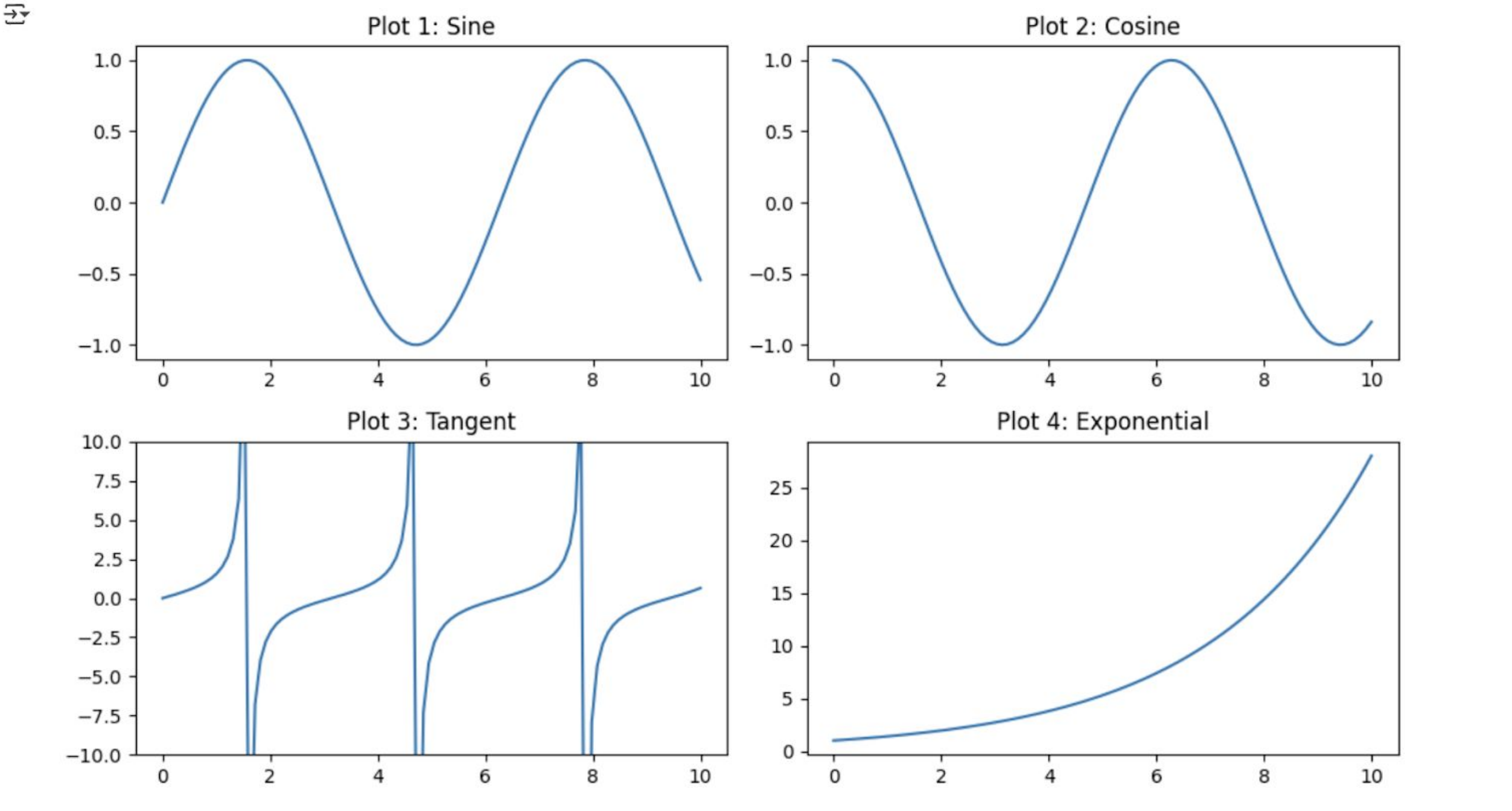
axes[0, 0].plot(x, np.sin(x))
axes[0, 0].set_title("Plot 1: Sine")

axes[0, 1].plot(x, np.cos(x))
axes[0, 1].set_title("Plot 2: Cosine")

axes[1, 0].plot(x, np.tan(x))
axes[1, 0].set_ylim(-10, 10)
axes[1, 0].set_title("Plot 3: Tangent")

axes[1, 1].plot(x, np.exp(x / 3))
axes[1, 1].set_title("Plot 4: Exponential")

plt.tight_layout()
plt.show()
```


Plot 1: Sine
Plot 2: Cosine
Plot 3: Tangent
Plot 4: Exponential

4️⃣ Shared Axes Subplots Use `sharex=True, sharey=True` to align all subplots on the same scale.

Shared Axes Subplots Example

Purpose Show how to create subplots that share x and/or y axes so that zooming or axis limits line up across subplots. Useful for direct visual comparisons.

Key syntax

- `fig, ((ax1, ax2), (ax3, ax4)) = plt.subplots(2, 2, sharex=True, sharey=True, figsize=(...))`
  - `sharex=True` makes all subplots share the same x-axis limits and ticks.
  - `sharey=True` makes them share the same y-axis limits and ticks.
- `fig.suptitle("...")` — set an overall (figure-level) title.
- `plt.tight_layout(rect=[0, 0, 1, 0.95])` — reserve space for the suptitle.

Line-by-line explanation

1. Create `x` and the 2×2 axes, with `sharex` and `sharey` enabled.
2. Plot functions on each axis and set individual titles.
3. Label shared x- and y-axes selectively to avoid clutter (e.g., only bottom row or left column).
4. Give the full figure a `suptitle` and call `tight_layout`.

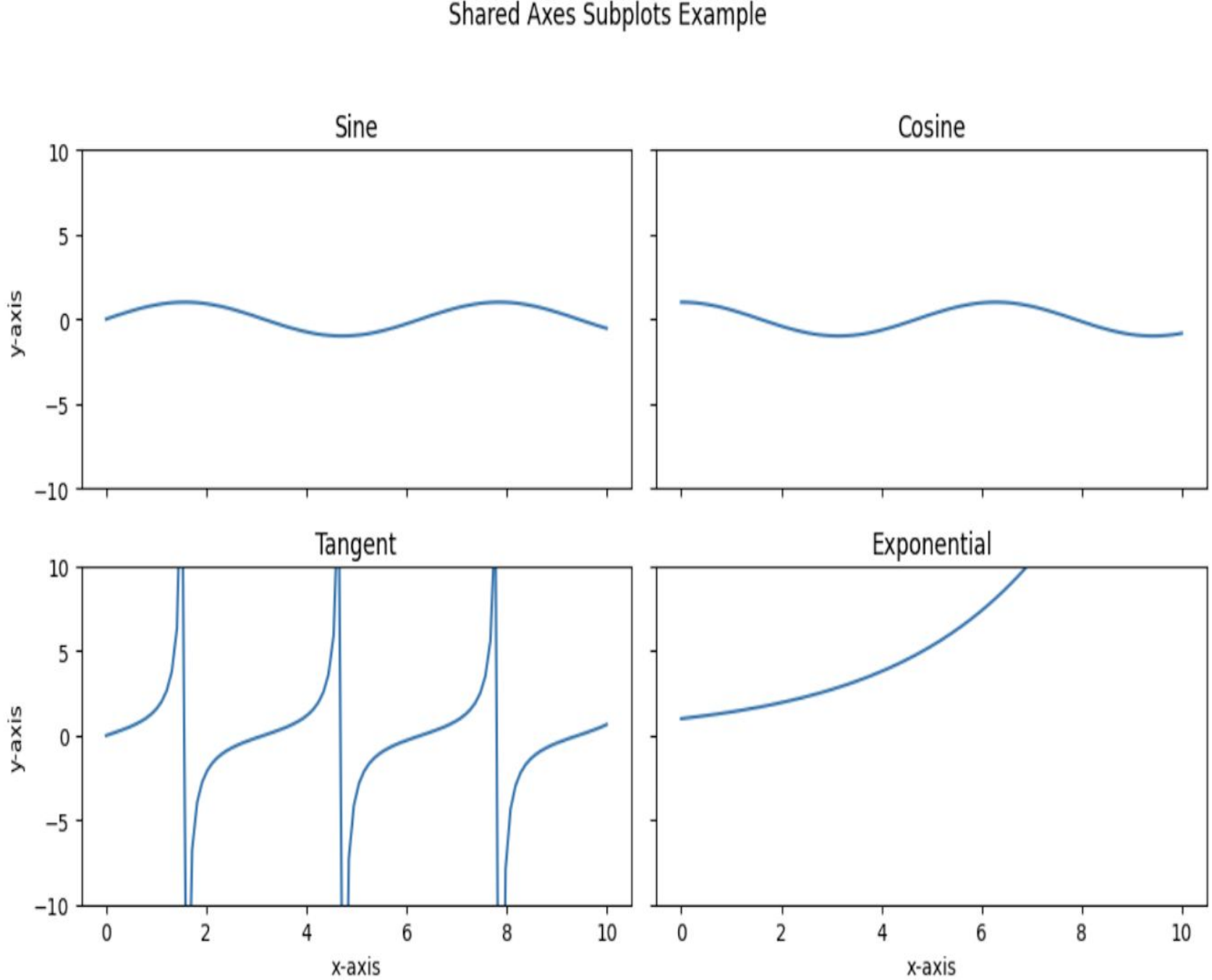
```
x = np.linspace(0, 10, 100)
fig, ((ax1, ax2), (ax3, ax4)) = plt.subplots(2, 2, figsize=(10, 6), sharex=True, sharey=True)

ax1.plot(x, np.sin(x));  ax1.set_title("Sine")
ax2.plot(x, np.cos(x));  ax2.set_title("Cosine")
ax3.plot(x, np.tan(x));  ax3.set_ylim(-10, 10); ax3.set_title("Tangent")
ax4.plot(x, np.exp(x / 3)); ax4.set_title("Exponential")

ax3.set_xlabel("x-axis")
ax4.set_xlabel("x-axis")
ax1.set_ylabel("y-axis")
ax3.set_ylabel("y-axis")

fig.suptitle("Shared Axes Subplots Example")
plt.tight_layout(rect=[0, 0, 1, 0.95])
plt.show()
```

[4]


Shared Axes Subplots Example
Sine
Cosine
Tangent
Exponential
y-axis
x-axis
10
5
0
−5
−10
0
2
4
6
8
10

Advanced Layout with GridSpec — Spanning Columns

Purpose Demonstrate `GridSpec`-style layout to create an arrangement where one subplot spans multiple columns while others fill remaining grid slots.

Key syntax

- `fig = plt.figure(figsize=(...))`
- `gs = fig.add_gridspec(nrows, ncols)` — create a grid spec object.
- `fig.add_subplot(gs[0, :])` — span the top row across all columns.
- `fig.add_subplot(gs[1, 0])`, `fig.add_subplot(gs[1, 1])` — bottom-left and bottom-right.

Line-by-line explanation

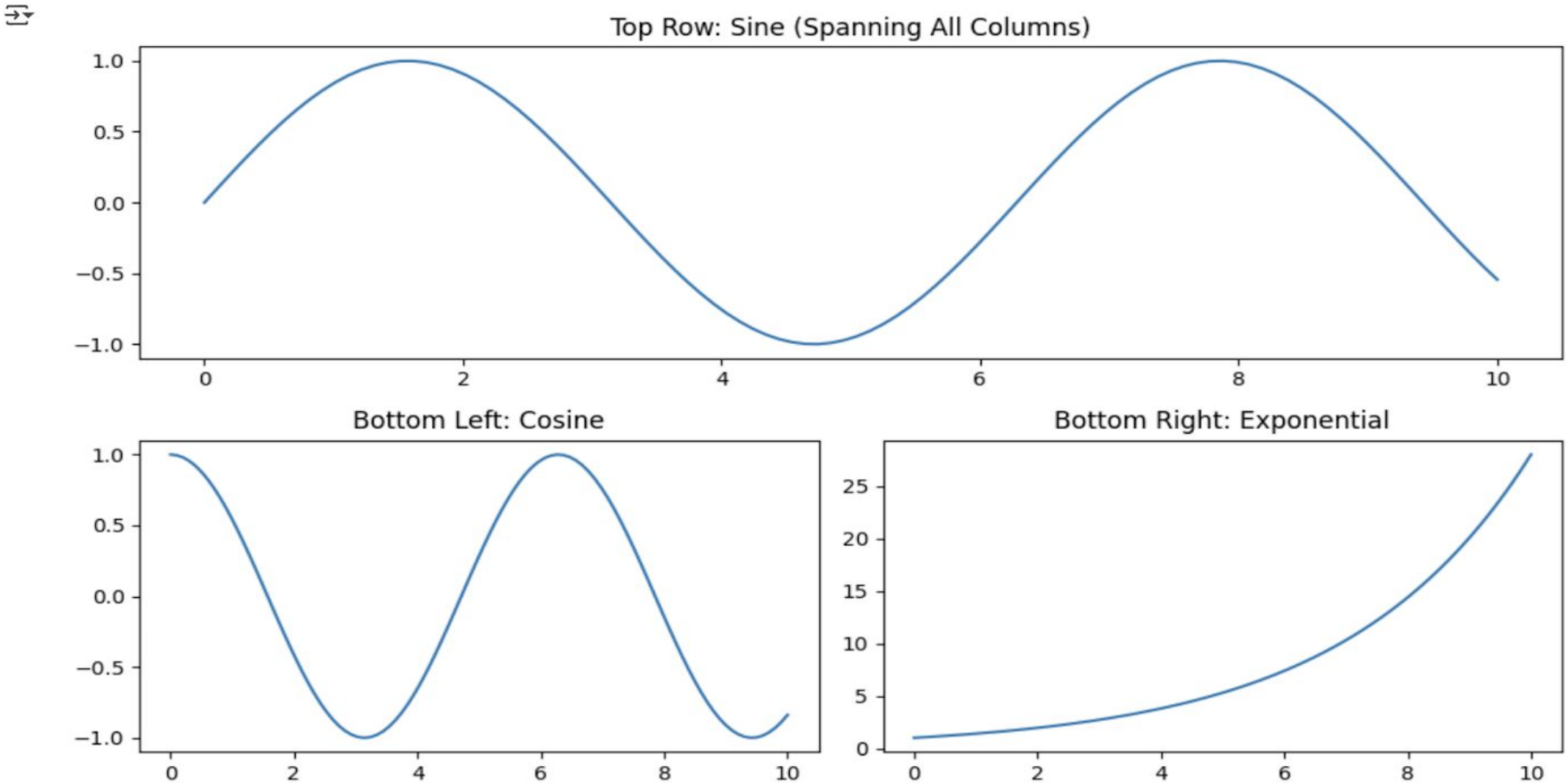
1. `gs = fig.add_gridspec(2, 2)` — create a 2×2 layout grid.
2. `ax1 = fig.add_subplot(gs[0, :])` — full-width top plot.
3. Create the bottom two plots separately and set titles.
4. Call `plt.tight_layout()` to tidy spacing.

```
x = np.linspace(0, 10, 100)
fig = plt.figure(figsize=(10, 6))
gs = fig.add_gridspec(2, 2)

ax1 = fig.add_subplot(gs[0, :])
ax2 = fig.add_subplot(gs[1, 0])
ax3 = fig.add_subplot(gs[1, 1])

ax1.plot(x, np.sin(x)); ax1.set_title("Top Row: Sine (Spanning All Columns)")
ax2.plot(x, np.cos(x)); ax2.set_title("Bottom Left: Cosine")
ax3.plot(x, np.exp(x / 3)); ax3.set_title("Bottom Right: Exponential")

plt.tight_layout()
plt.show()
```



Saving Figures `savefig()` saves the figure to an image file (PNG, JPG, PDF, etc.)

Syntax:

```
plt.savefig("filename.png", dpi=300, bbox_inches='tight')
```

Heatmap / 2D Intensity Map with Custom Colormap

Purpose Create a smooth heatmap (image) from a function defined on a 2D grid and demonstrate customizing color maps and adding a colorbar.

Key syntax

- `np.meshgrid(x, y)` — create 2D coordinate arrays from 1D arrays.
- `ax.imshow(Z, origin='lower', cmap=..., extent=[x_min, x_max, y_min, y_max], interpolation='bicubic')`
  - `origin='lower'` places the (0,0) corner at bottom-left.
  - `cmap` accepts either built-in colormap names or a custom `LinearSegmentedColormap`.
  - `extent` maps image array indices to real coordinate ranges.
- `fig.colorbar(heatmap, ax=ax)` — add a color scale legend.

Custom colormap

- `mcolors.LinearSegmentedColormap.from_list(name, colors)` constructs a gradient colormap from a list of color hex codes.

Line-by-line explanation

1. Define helper functions `distance` and `radiation` to compute intensity at (x,y).
2. Create `X, Y` grid via `np.meshgrid`.
3. Compute `Z = radiation(X, Y)` which holds intensity values.
4. Build a custom colormap and call `ax.imshow()` to render `Z`.
5. Add title, axis labels, and a colorbar that explains color→value mapping.

```python
import matplotlib.pyplot as plt
import matplotlib.colors as mcolors
import numpy as np

# Define distance and radiation functions
def distance(x1, y1, x2, y2):
    return np.sqrt((x1 - x2)**2 + (y1 - y2)**2)

def radiation(x, y):
    k = 0.2
    return (100*np.exp(-k*distance(x, y, 5, 5)) +
            120*np.exp(-k*distance(x, y, 25, 6)) +
            80*np.exp(-k*distance(x, y, 10, 20)) +
            90*np.exp(-k*distance(x, y, 22, 24)) +
            110*np.exp(-k*distance(x, y, 15, 15)))

# Create grid points
points = 150
x = np.linspace(0, 30, points)
y = np.linspace(0, 30, points)
X, Y = np.meshgrid(x, y)
Z = radiation(X, Y)

# Define custom colormap (blue → red)
colors = ["#0000FF", "#00FFFF", "#00FF00", "#FFFF00", "#FF0000"]
cmap = mcolors.LinearSegmentedColormap.from_list("custom_cmap", colors)

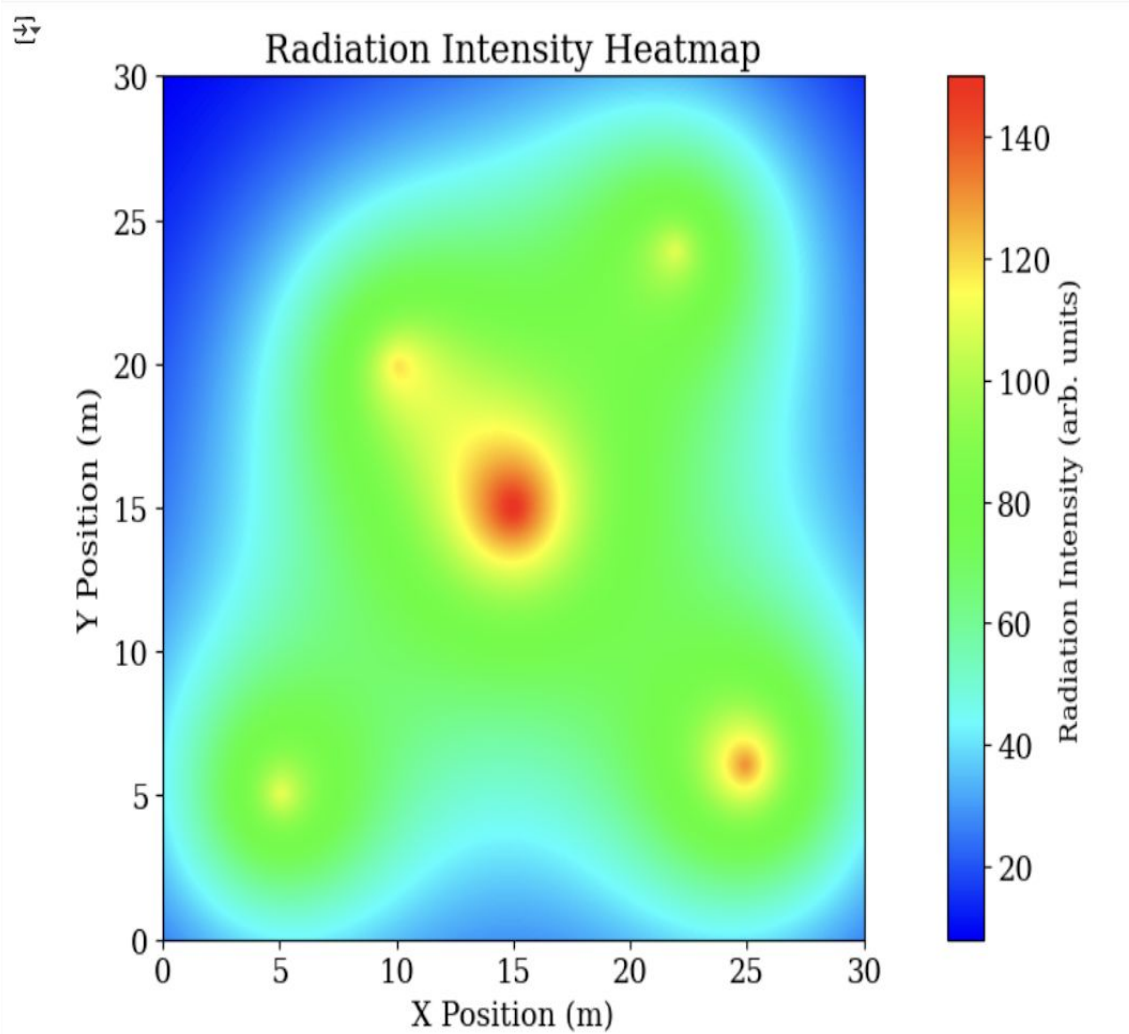
# ✅ Use a universal font (DejaVu Serif) to avoid warnings
plt.rcParams['font.family'] = 'DejaVu Serif'
plt.rcParams['font.size'] = 14

# Plot heatmap
fig, ax = plt.subplots(figsize=(12, 6))
heatmap = ax.imshow(Z, origin='lower', cmap=cmap,
                    extent=[0, 30, 0, 30], interpolation='bicubic')

ax.set_title("Radiation Intensity Heatmap")
ax.set_xlabel("X Position (m)")
ax.set_ylabel("Y Position (m)")

# Add colorbar
cbar = fig.colorbar(heatmap, ax=ax)
cbar.set_label("Radiation Intensity (arb. units)", fontsize=12)

plt.tight_layout()
plt.show()
```



🖌️8️⃣ Set Font Name and Font Size Globally

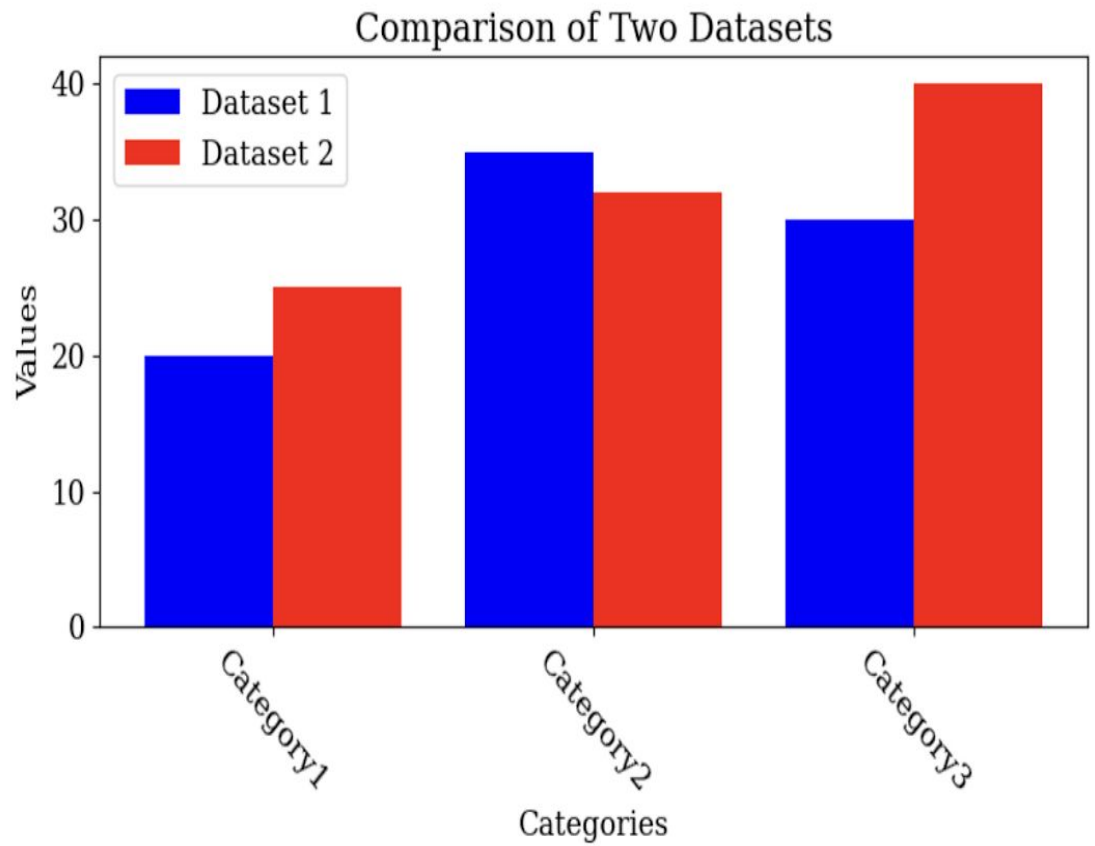
```
import matplotlib.pyplot as plt
plt.rcParams['font.family'] = 'Times New Roman'
plt.rcParams['font.size'] = 14
```

📊9️⃣ Side-by-Side Bar Chart

```
categories = ['Category1', 'Category2', 'Category3']
x = np.arange(len(categories))
dataset1 = [20, 35, 30]
dataset2 = [25, 32, 40]
width = 0.4

fig, ax = plt.subplots(figsize=(8, 5))
ax.bar(x - width/2, dataset1, width=width, label='Dataset 1', color='blue')
ax.bar(x + width/2, dataset2, width=width, label='Dataset 2', color='red')

ax.set_xlabel('Categories')
ax.set_ylabel('Values')
ax.set_title('Comparison of Two Datasets')
ax.set_xticks(x)
ax.set_xticklabels(categories, rotation=-45)
ax.legend()
plt.tight_layout()
plt.show()
```



🖼️🔟 Image Plotting

[ ]

```python
import matplotlib.pyplot as plt
import numpy as np

# Example 1: Random Image
image = np.random.rand(100, 100, 3)
plt.imshow(image)
plt.title("Random Color Image")
plt.axis('off')
plt.show()

# Example 2: Load External Image
import matplotlib.image as mpimg
# img = mpimg.imread('your_image.png')
# plt.imshow(img)
# plt.axis('off')
# plt.title("Loaded Image")
# plt.show()
```



3D Line Plot using mpl_toolkits.mplot3d

Purpose Show how to plot a 3D line (parametric curve) by enabling the 3D projection of an Axes.

Key syntax

- `from mpl_toolkits.mplot3d import Axes3D` — registers 3D projection (often not explicitly used but required).
- `fig.add_subplot(111, projection='3d')` — create 3D axes.
- `ax.plot(x, y, z)` — plot a 3D curve, where x, y, z are same-length 1D arrays.

Line-by-line explanation

1. Create `z = np.linspace(0, 10, 100)` as the parameter.
2. Compute x = sin(z), y = cos(z) — parametric helix-like curve.
3. Make a figure, add a 3D subplot, plot x,y,z, set title, and display.

[ ]

```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import numpy as np

z = np.linspace(0, 10, 100)
x = np.sin(z)
y = np.cos(z)

fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.plot(x, y, z)
ax.set_title("3D Line Plot")
plt.show()
```

